

llama.cpp Master API Reference

Unified OpenAI-Compatible Documentation for Java Edge Services (2026)

1. Connection Details

Base URL: `http://localhost:8080/v1`

POST `/chat/completions`

Main endpoint for generating chat responses with support for streaming, tool use, and advanced sampling.

2. Request Schema

Field	Type	Description
<code>messages</code>	Array	Required. List of message objects <code>{"role": "user", "content": "..."} </code>
<code>stream</code>	Boolean	Enable Server-Sent Events (SSE). Recommended for Edge UIs.
<code>temperature</code>	Float	Randomness (0.0 - 2.0). Default: 0.8.
<code>min_p</code>	Float	Minimum probability relative to top token. Highly stable sampler.
<code>xtc_threshold</code>	Float	XTC Exclude Top Choices (>0.0). Reduces "AI repetitive" phrasing.
<code>dry_multiplier</code>	Float	DRY Don't Repeat Yourself penalty. Prevents loops.
<code>max_tokens</code>	Integer	Limit output length. (Alias: <code>n_predict</code>).
<code>samplers</code>	String	Execution order of samplers (e.g., <code>"dkypmxt"</code>).
<code>cache_prompt</code>	Boolean	Keeps KV cache in memory for near-instant multi-turn replies.

Field	Type	Description
<code>grammar</code>	String	GBNF grammar to force specific formats (JSON/CSV).

3. Payload Examples

Request JSON

```
{ "messages": [ {"role": "system",  
  "content": "You are a helpful agent."},  
  {"role": "user", "content": "What is  
2+2?"} ], "temperature": 0.7, "stream":  
false, "cache_prompt": true }
```

Response JSON

```
{ "id": "chatcmpl-123", "choices": [{  
  "index": 0, "message": { "role":  
    "assistant", "content": "2 + 2 is 4." },  
  "finish_reason": "stop" }], "usage": {  
  "prompt_tokens": 15,  
  "completion_tokens": 8, "total_tokens":  
    23 } }
```

4. Java Implementation (HttpClient + GSON)

Optimized for edge: Reuses HttpClient, parses JSON into objects via GSON.

```
import com.google.gson.Gson; import java.net.URI; import java.net.http.*; public  
class LlamaService { private static final HttpClient CLIENT =  
  HttpClient.newHttpClient(); private static final Gson GSON = new Gson(); public  
  String ask(String url, Object payload) throws Exception { var req =  
    HttpRequest.newBuilder() .uri(URI.create(url + "/chat/completions"))  
    .header("Content-Type", "application/json")  
    .POST(HttpRequest.BodyPublishers.ofString(GSON.toJson(payload))) .build(); var res =  
    CLIENT.send(req, HttpResponse.BodyHandlers.ofString()); // Simple GSON navigation  
    var obj = GSON.fromJson(res.body(), com.google.gson.JsonObject.class); return  
    obj.getAsJsonArray("choices").get(0).getAsJsonObject() .getAsJsonObject("message").get(
```

5. Edge Optimization Checklist

- **Memory:** Use `min_p` and `xtc_threshold` to maintain quality at lower bit-depth (4-bit/ Q4_K_M) quantizations.
- **Speed:** Always enable `cache_prompt: true` for agents to avoid re-calculating the entire chat history.

- **Footprint:** Avoid LangChain4j for simple agents; stick to the `HttpClient + GSON` stack to keep .jar size under 1MB.
- **Native:** Use GraalVM to compile your Java service into a native binary for 10ms startup times.